

# SPRING CORE & SPRING BOOT MASTER NOTES

*Comprehensive Technical Reference Guide*

## Module 1: Dependency Injection & Loose Coupling Core Concepts

### What is Tight Coupling?

Tight coupling occurs when a class has a direct, hardcoded dependency on another concrete class. When Class A creates an instance of Class B internally using the `new` keyword, Class A cannot function, be tested, or be reused without Class B.

### Why Directly Creating Dependencies Inside a Class is a Problem

- **Inflexibility:** Changing the underlying dependency implementation requires modifying the code of the dependent class.
- **Untestability:** You cannot isolate the class under test. It is impossible to swap the real dependency with a mock or stub object during unit testing.
- **Violates SRP (Single Responsibility Principle):** The class is responsible for both its primary business logic and managing the lifecycle of its dependencies.

### How Interfaces Help in Loose Coupling

Interfaces introduce an abstraction layer between the dependent class and its dependency. Instead of depending on a concrete implementation (e.g., `MySQLDatabase`), the class depends on an abstraction interface (e.g., `Database`). This allows any implementation matching the contract to be supplied seamlessly without modifying the dependent class.

### Why Interface Alone is Not Enough

Even if a class declares an instance variable using an interface type, if it still executes `Database db = new MySQLDatabase();` inside its constructor, tight coupling remains at the instantiation point. The class still explicitly references the concrete class name, meaning an external mechanism is needed to pass (inject) the concrete implementation.

### What is Dependency Injection (DI)?

Dependency Injection is a design pattern that eliminates hardcoded dependency creation. Instead of a class instantiating its own dependencies, the dependencies are provided ("injected") into the class from an external entity (the injector or framework container).

## What is Inversion of Control (IoC)?

Inversion of Control is a broad software engineering paradigm where the control flow of a program is inverted. In traditional programming, your application code controls instantiation and calls framework methods. Under IoC, the framework controls the orchestration and lifecycle, driving and invoking your application code instead.

## Relationship Between IoC and DI

**Paradigm vs. Implementation:** IoC is the high-level philosophy or principle (inverting control of lifecycle/flow), while DI is the specific design pattern used to implement IoC specifically for object dependency management.

## Constructor Injection vs. Setter Injection

| Criteria         | Constructor Injection                                                             | Setter Injection                                                      |
|------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Immutability     | Supports <code>final</code> fields. Fields cannot be altered post-initialization. | Cannot use <code>final</code> . Fields can be mutated anytime.        |
| Dependency State | Ensures dependencies are non-null and fully initialized at object creation.       | Objects can be created in an incomplete, partially initialized state. |
| Use Case         | Mandatory, core dependencies.                                                     | Optional, secondary configurations or changeable dependencies.        |

## What is the Spring IoC Container?

The Spring IoC Container is the core engine of the framework. It manages the complete lifecycle of application objects from instantiation, configuration, and dependency resolution to destruction. It reads configuration metadata (annotations, Java code, or XML) to assemble the runtime system.

## What are Spring Beans?

A Spring Bean is simply an object that is instantiated, assembled, and managed by the Spring IoC Container. It is a standard Java object, but registered inside the framework context.

## How Spring Manages Object Creation and Dependency Wiring

Spring utilizes a reflection-driven mechanism. It scans the designated packages, parses metadata, generates metadata descriptors ( `BeanDefinition` ), instantiates the objects via constructors, detects injection points, and hooks dependencies together automatically.

## Module 2: Spring Container Internals & Dependency Resolution

### What is ApplicationContext?

`ApplicationContext` is the advanced interface representing the Spring IoC container. It extends `BeanFactory` and adds enterprise-level features such as internationalization, event publishing, and transparent integration with application layers.

### How AnnotationConfigApplicationContext Works

It is a concrete implementation of `ApplicationContext` that accepts Java configuration classes or annotated components as input. Upon initialization, it processes `@Configuration` classes and registers all contained bean definitions via reflection.

### Why We Use @Configuration

The `@Configuration` annotation indicates that a class contains factory methods annotated with `@Bean`. Crucially, Spring wraps these classes in a CGLIB proxy to ensure that consecutive calls to factory methods return the same cached singleton bean instance, enforcing container scope rules.

### How @ComponentScan Finds Classes & How @Component Creates Beans

`@ComponentScan` tells Spring which base packages to scan. The scanner looks for classes marked with stereotype annotations like `@Component`, `@Service`, or `@Repository`. For each discovered class, Spring automatically builds a `BeanDefinition` and schedules it for object creation.

### What Happens When We Call getBean()?

When `getBean()` is invoked, Spring checks its internal singleton cache map ( `singletonObjects` ) for an existing instance matching the name/type. If found, it returns the instance; if not, and the scope is singleton, it initiates the bean creation pipeline on demand.

### Constructor, Field, and Setter Injection Comparison

- **Constructor Injection:** Dependencies are passed via the constructor. *Highly preferred because it guarantees object safety, immutability, and simple testing without container mocking.*
- **Setter Injection:** Dependencies are passed via public setter methods.
- **Field Injection:** Dependencies are injected directly into private fields using reflection (via `@Autowired`). *Discouraged because it hides dependencies, violates encapsulation, and couples code tightly to a container.*

### How Spring Resolves Dependencies Internally

Spring matches dependencies primarily **by type**. If exactly one bean matches the type required at an injection point, it wires it. If multiple match, it attempts to resolve the ambiguity **by name**.

## Handling Resolution Failures

- **No Matching Bean:** Throws `NoSuchBeanDefinitionException` at startup.
- **Multiple Beans of Same Type:** Throws `NoUniqueBeanDefinitionException` unless resolved.

## How @Primary and @Qualifier Work

- `@Primary` specifies a fallback preference when multiple beans match by type. The marked bean takes precedence.
- `@Qualifier` overrides all fallback behavior by specifying the exact name/ID of the target bean to wire at the specific injection point.

## Difference Between @Component and @Bean

| Feature       | @Component                         | @Bean                                                 |
|---------------|------------------------------------|-------------------------------------------------------|
| Target        | Applied to class level.            | Applied to method level within configuration classes. |
| Control       | Used when you own the source code. | Used to wrap third-party library classes into beans.  |
| Customization | Implicit naming/wiring.            | Explicit control over initialization logic.           |

## Module 3: Advanced Core: Scopes, Lifecycle & Circular Dependencies

### What is Circular Dependency?

A circular dependency occurs when Class A requires an instance of Class B, and Class B simultaneously requires an instance of Class A. In plain Java, trying to construct both via constructors leads to a stack overflow or compilation block.

### Constructor vs. Setter/Field Injection Circular Dependency

**The Constructor Trap:** Spring cannot resolve a circular dependency if both beans use Constructor Injection because neither instance can be instantiated to be passed to the other, resulting in a `BeanCurrentlyInCreationException`.

For Setter or Field injection, Spring breaks the cycle using its **early reference concept**. It instantiates the raw object via its empty constructor, exposes a reference to it via a factory cache (three-level cache) before initialization is complete, and then wires the dependency later.

## Why Circular Dependency is a Design Smell

It indicates poor architecture, tight architectural coupling, and an implicit violation of the Single Responsibility Principle. It usually means a third coordinating component should extract the shared logic.

## Spring Bean Scopes

- **Singleton (Default):** Creates exactly one instance per container. Cached and shared globally.
- **Prototype:** Creates a completely new bean instance every single time it is requested from the container.
- **Web Scopes:** `request` (one per HTTP request), `session` (one per HTTP session), and `application` (one per ServletContext context).

## Prototype Bean Inside a Singleton Bean Problem

Because a Singleton bean is initialized only once, any dependencies injected into it during creation are also wired only once. If you inject a Prototype bean into a Singleton bean, that Prototype bean remains static forever inside the Singleton, defeating its multi-instance purpose. To fix this, you must use **Method Injection** (`@Lookup`) or inject the `ApplicationContext` directly.

## Eager vs. Lazy Initialization

By default, singleton beans are initialized **eagerly** at container startup to discover failures early. `@Lazy` defers bean instantiation until the very first time it is explicitly called at runtime. You can also enforce global lazy initialization in Spring Boot via `spring.main.lazy-initialization=true`.

---

# Module 4: Deep Dive into Spring Bean Lifecycle & Infrastructure

## What is a BeanDefinition?

A `BeanDefinition` is an internal recipe object containing full configuration metadata for a bean (class name, scope, init methods, factory methods, constructor arguments, property values). Spring processes configuration files first to create these metadata recipes before instantiating objects.

## Instantiation vs. Initialization

- **Instantiation:** Creating the raw Java object instance in heap memory (equivalent to executing `new` via constructor reflection).
- **Initialization:** Occurs *after* properties and dependencies are injected. This is where customized startup logic runs, configuration verification happens, and proxy wrappers are created.

## The Complete Lifecycle Pipeline Steps

1. Parse configuration and build `BeanDefinition` definitions.

2. **Instantiation:** Raw object created via reflection.
3. **Populate Properties:** Spring resolves and injects all dependencies (DI).
4. **Aware Interfaces Trigger:** Calls `BeanNameAware` , `BeanFactoryAware` , and `ApplicationContextAware` .
5. **BeanPostProcessor (Before Initialization):** `postProcessBeforeInitialization()` processes custom hooks like `@PostConstruct` .
6. **Initialization Callbacks:** Runs `InitializingBean.afterPropertiesSet()` followed by custom `init-method` definitions.
7. **BeanPostProcessor (After Initialization):** `postProcessAfterInitialization()` creates AOP proxies if required. Bean is ready for use.
8. **Destruction:** When container closes, `@PreDestroy` , `DisposableBean.destroy()` , and custom `destroy-method` execute.

**Prototype Lifecycle Exception:** Spring does not track or call destruction methods for Prototype scoped beans. The client code must manage cleanup and release any held resources manually.

## Module 5: Legacy Metadata & XML Configuration

### Why XML Exists & Core Mechanics

Historically, XML separated configuration metadata cleanly from business source code. Spring leverages a reader (like `XmlBeanDefinitionReader` ) to convert `<bean>` tags into standard internal `BeanDefinition` objects.

### Core Directives

- `ClassPathXmlApplicationContext` loads XML metadata configuration files from the application classpath.
- **id vs. name:** `id` enforces strict alphanumeric structural uniqueness checked by XML parsers. `name` allows special characters, spaces, and multi-alias comma-separated lists.
- **value vs. ref:** `value` injects primitive values or literal strings; `ref` points to another declared bean instance identifier.

### XML Autowiring Modes

- `byName` : Finds a bean matching the exact property variable name.
- `byType` : Finds a bean matching the exact data type contract.
- `constructor` : Matches constructor arguments by type alignment.

**Hybrid Configuration:** XML and annotations work together via `<context:component-scan>` configurations inside XML files.

---

## Module 6: Spring Boot: Automation, Auto-Configuration & Bootstrapping

### Why Spring Boot After Spring Core?

Spring Core requires boilerplate setups (view resolvers, embedded servers, database pools, transaction configurations). Spring Boot introduces opinionated defaults, embedded environments, and rapid developer automation.

### Deconstructing `@SpringBootApplication`

It is a convenience meta-annotation that wraps three core annotations:

- `@SpringBootConfiguration` : Designates the main class as a Spring configuration source.
- `@ComponentScan` : Activates automated component discovery starting from the class package downwards.
- `@EnableAutoConfiguration` : The engine that analyzes the classpath to dynamically add missing configurations automatically.

### Auto-Configuration Mechanics

Boot scans dependencies using conditional evaluations defined in `META-INF/spring.factories` or auto-configuration classes:

- `@ConditionalOnClass` : Activates a component only if a specific class is verified on the current system classpath.
- `@ConditionalOnMissingBean` : Registers a default fallback bean only if developers haven't supplied their own custom bean definition.

### What Happens inside `SpringApplication.run()`?

It bootstraps the application: launches a stopwatch, spins up appropriate web contexts (Reactive, Servlet, or standard), runs the infrastructure scanners, configures the underlying platform environment, creates the IoC container, and executes startup runners.

### Runner Interfaces (`CommandLineRunner` & `ApplicationRunner`)

Instead of manually hacking logic into the static `main()` thread before initialization, these interfaces allow safe code execution right after the context is fully loaded.

- `CommandLineRunner` provides raw access to arguments as a basic array of strings ( `String... args` ).

- `ApplicationRunner` wraps the arguments inside a higher-level `ApplicationArguments` object, allowing parsed option names and values to be read directly.
- 

## Module 7: Externalized Configuration & Properties

### Why Externalized Configuration Matters

Hardcoded settings prevent code from moving across environments. Externalization decouples environment data (URLs, passwords, ports) from the binary bundle.

### Property Strategies

Spring Boot processes `application.properties` or `application.yml` natively. Properties can be injected using:

- `@Value` : Useful for quick standalone values (e.g., `@Value("${app.timeout:5000})"`). *Drawback: Becomes verbose, messy, and cannot be cleanly structured if managing large blocks of related properties.*
- `@ConfigurationProperties` : Provides strongly-typed, object-oriented group mapping. Supports type validation, IDE auto-completion, and nested hierarchies.

### Relaxed Binding

Spring Boot allows flexible property naming variations. For example, a property declared as `serverTimeout` in a Java object can match `server-timeout`, `server_timeout`, or `SERVER_TIMEOUT` inside properties files transparently.